Mathematical Modeling Tutorial

Differential Equation

Dylan He

1 Modeling by Differential Equation

Differential equations are widely applied to modern sciences. Generally speaking, differential equations express the relationship between some variables and their change rates. For the natural sciences in particular, differential equations are perhaps the most important mathematical tool. You are already familiar with it because of your math courses. So we will mainly discuss how to describe real phenomena in terms of differential equations and how to solve them.
The main steps for modeling by differential equations are shown below.

- Analyzing the relative variables.
- Analyzing the question.
- Making assumptions.
- Finding relationship and complete differential equations.
- Solving them analytically or numerically.
- verifying your models and modifying them.

Among all the steps, I guess the "finding a relationship and complete differential equations." confuses you the most. I will give some examples of it.

1.1 Physical Rules

This is the easiest way (maybe?) to model because you can utilize existing equations. In that case, the tasks can be shifted to applying physical rules.

Here's an example.

Decay of Radioactive Materials
Physicists told us that the rate of decay for a radioactive substance is proportional to its amount of substance. Therefore, we know that there will be two variables in the equation, which are "the rate of decay" and "amount of substance". We know that the rate of decay can be expressed as the change rate of the amount of substance, which is $\frac{\mathrm{d}N}{\mathrm{d}t}$. So we can derive the equation.

$$\frac{\mathrm{d}N}{\mathrm{d}t} = -\lambda N \tag{1}$$

Where the constant *lambda* depends on both how we define variables and the actual data. It's easy to find its analytical solution.

$$N = N_0 e^{-\lambda t} \tag{2}$$

That's a very basic and easy process of mathematical modeling. When the problems are highly relative to some physical mechanism, using physical rules is the best choice. Especially some differential equations have already been completed by physicists. Another example is heat conduction problems, we will discuss them later.

1.2 Existing Equations Systems

You have already known that differential equations are widely used. It's no surprise to find that some phenomena in biology or sociology can be explained by differential equations. A typical is the Lotka-Volterra model. Let's take my model in the 2024 American Mathematical Contest In Modeling as an example.
Lotka-Volterra Model
Lampreys is a kind of ugly fish like some Cthulhu monsters. Assuming it has no natural predators. And species A is its prey. In biology, the Lotka-Volterra model is used to predict the population of a pair of predators and prey.

$$\frac{\mathrm{d}U}{\mathrm{d}t} = \alpha U - \gamma UV \tag{3}$$

$$\frac{\mathrm{d}V}{\mathrm{d}t} = -\beta V + e\gamma UV \tag{4}$$

Where α, β, γ, and e are four positive constants that determine the behavior of the model. We can explain those constants according to our context. α

is the natural growing rate of A, and β is the mortality rate of lamprey. γ is the possibility of A being eaten by lampreys, and e is a transfer factor. This model is nonlinear and it has no analytical solution. By this example, you can see how we can be inspired by existing models. In that case, finding suitable models quickly is the key point of solving problems.

1.3 Analyzing the variables

We may encounter some problems without any existing models. What can we do? The only way feasible is to analyze the relationship between variables and derive the model. From the previous examples, you can find that the differential equations can always be written as follows.

$$\frac{\mathrm{d}y}{\mathrm{d}x} = f(x, y) \tag{5}$$

Our task is just to find $f(x, y)$ and combine all equations. Let's look at a classic pestilence problems.

SEIDR Model

In a region, a kind of deadly epidemic is on the rise. To simulate the development of the epidemic, we divide the total population into 5 parts. S group (Susceptible), E group (Exposed), I group (Infectious), D group (Died), and R group (Recovered). We can use state transfer figures to express the process. Let's analyze them one by one. For S, its change rate comes from

Figure 1: State Transfer Figure for SEIDR

being infected due to E, I, and social touching intensity. From "intuition", the infected number equals the total number of S times the social touching intensity and times the possibility of encountering I and E people. That is $-\beta_1 S\frac{I}{N'} - \beta_2 S\frac{E}{N'}$, where $N' = S + E + I + R$. For E, it will increase because S people are infected and become E but also decrease because E people will

turn into I people after some time. So it can be written as $\frac{\mathrm{d}E}{\mathrm{d}t} = \frac{\mathrm{d}S}{\mathrm{d}t} - \sigma E$. E people will become I people and I people will die or recover. So the change rate equals $\frac{\mathrm{d}I}{\mathrm{d}t} = \sigma E - \gamma I - \theta I$. In conclusion, we can derive the differential equations system.

$$\frac{\mathrm{d}S}{\mathrm{d}t} = -\beta_1 S\frac{I}{N'} - \beta_2 S\frac{E}{N'} \tag{6}$$

$$\frac{\mathrm{d}E}{\mathrm{d}t} = \beta_1 S\frac{I}{N'} + \beta_2 S\frac{E}{N'} - \sigma E \tag{7}$$

$$\frac{\mathrm{d}I}{\mathrm{d}t} = \sigma E - \gamma I - \theta I \tag{8}$$

$$\frac{\mathrm{d}D}{\mathrm{d}t} = \theta I \tag{9}$$

$$\frac{\mathrm{d}R}{\mathrm{d}t} = \gamma I \tag{10}$$

This system can't have an analytical solution. We will discuss the numerical method later.

2 Analysis and solution of differential equations

2.1 Integral Transform

Integral transform, including Fourier Transform, Laplace Transform, Mellin Transform, etc., it's a strong tool to simplify differential equations.

2.1.1 Laplace Transform

In this subsection, I will mainly introduce the Laplace Transform. Mathematically, it's defined as follows.

$$F(x) = \int_0^\infty e^{-\mathrm{st}} f(t)\mathrm{d}t \tag{11}$$

Where the condition is that improper integral on the right-hand side must converge. We call $F(x)$ the Laplace Transform of $f(x)$, which is written as $F(x) = \mathcal{L}(f(x))$. By applying integral by parts, we can derive the following formula:

$$\mathcal{L}(f'(s)) = sF(x) - f(0) \tag{12}$$

$$\mathcal{L}(f''(s)) = s^2F(x) - sf(0) - f'(0) \tag{13}$$

From those formulas, we can see the initial conditions are needed. Laplace Transform has the ability to transform differential equations into algebraic equations. Considering the example:

$$\frac{\mathrm{d}^2y}{\mathrm{d}x^2} = \frac{\mathrm{d}y}{\mathrm{d}x} + e^{-x} \tag{14}$$

$$y(0) = 1, y'(0) = 0 \tag{15}$$

Previously, we needed to find the solution to the homogeneous equation and combine it with the particular solution of the equation to form the general solution. In this example, the exponent of the input function happens to be the root of the characteristic equation, so we need to use the exponential shift method to find the particular solution. We also need to substitute the initial value conditions to determine the parameters in the general solution. The solution involves quite a few steps from start to finish, while the Laplace transform is relatively simple. Take the L.T. on both sides.

$$s^2Y - s - Y = \frac{1}{s+1} \tag{16}$$

Using elementary school knowledge, we can get Y.

$$Y = \frac{s^2+s+1}{(s+1)^2(s-1)} = \frac{-1}{2}\frac{1}{(s+1)^2} + \frac{1}{4}\frac{1}{s+1} + \frac{3}{4}\frac{1}{s-1} \tag{17}$$

Take inverse L.T., we can get the solution:

$$y = \frac{1}{2}xe^{-x} + \frac{1}{4}e^{-x} + \frac{3}{4}e^{x} \tag{18}$$

Fortunately, there's no need to compute L.T. by hand. You can check the L.T. table for reference.

2.1.2 Fourier Transform

Intuitively, the Fourier transform converts a time domain signal into the frequency domain by decomposing the waveform into a superposition of many sinusoids of different frequencies. This transforms the study of the original function into the study of the amplitude and phase of the different frequency components.
It's defined as follows:

$$F(\omega) = \int_{-\infty}^{+\infty} f(t)e^{-i\omega t}\mathrm{d}t \tag{19}$$

表 5－1　常用拉普拉斯变换简表

公式号数	$f(t)=\mathscr{L}^{-1}\{F(s)\}$	$F(s)=\mathscr{L}\{f(t)\}$
1	$\delta(t)$	1
2	$\varepsilon(t)$	$\frac{1}{s}$
3	$t\varepsilon(t)$	$\frac{1}{s^2}$
4	$t^n\varepsilon(t)$	$\frac{n!}{s^{n+1}}$
5	$e^{at}\varepsilon(t)$	$\frac{1}{s-\alpha}$
6	$te^{at}\varepsilon(t)$	$\frac{1}{(s-\alpha)^2}$
7	$t^n e^{at}\varepsilon(t)$	$\frac{n!}{(s-\alpha)^{n+1}}$

Figure 2: Common Laplace transform formulas-1

续表

公式号数	$f(t)=\mathscr{L}^{-1}\{F(s)\}$	$F(s)=\mathscr{L}\{f(t)\}$
8	$\sin(\omega t)\varepsilon(t)$	$\frac{\omega}{s^2+\omega^2}$
9	$\cos(\omega t)\varepsilon(t)$	$\frac{s}{s^2+\omega^2}$
10	$\sinh(\beta t)\varepsilon(t)$	$\frac{\beta}{s^2-\beta^2}$
11	$\cosh(\beta t)\varepsilon(t)$	$\frac{s}{s^2-\beta^2}$
12	$e^{at}\sin(\omega t)\varepsilon(t)$	$\frac{\omega}{(s-\alpha)^2+\omega^2}$
13	$e^{at}\cos(\omega t)\varepsilon(t)$	$\frac{s-\alpha}{(s-\alpha)^2+\omega^2}$
14	$2re^{at}\cos(\omega t+\varphi)\varepsilon(t)$	$\frac{re^{j\varphi}}{s-\alpha-j\omega}+\frac{re^{-j\varphi}}{s-\alpha+j\omega}$
15	$\frac{1}{\omega_n\sqrt{1-\zeta^2}}e^{-\zeta\omega_n t}\sin(\omega_n\sqrt{1-\zeta^2})t\varepsilon(t)$	$\frac{1}{s^2+2\zeta\omega_n s+\omega_n^2}$

Figure 3: Common Laplace transform formulas-2

The Fourier transform, unlike the Laplace transform, can handle problems on the full plane and does not require time to be positive.
For the nth-order derivative, we can use F.T. to transform it into the product of coefficients.

$$\mathcal{F}(\frac{\mathrm{d}}{\mathrm{d}t}f(t)) = (i\omega)^n\mathcal{F}(t) \tag{20}$$

Similar to Laplace, this property is very helpful in solving differential equations. Consider the example of the wave function:

$$\frac{\partial^2 u}{\partial t^2} = c\frac{\partial^2 u}{\partial x^2} \tag{21}$$

with boundary conditions $u(x,t)|_{t=0} = \cos(x)$ and $\frac{\partial^2 u}{\partial t^2}|_{t=0} = \sin(x)$. Apply F.T., we derive an ordinary differential equation.

$$\frac{\partial^2 \hat{u}}{\partial t^2} = -c\omega^2\hat{u} \tag{22}$$

$$\mathcal{F}[\cos(x)] = \pi[\delta(\omega+1) + \delta(\omega-1)] \tag{23}$$
$$\mathcal{F}[\sin(x)] = i\pi[\delta(\omega+1) - \delta(\omega-1)] \tag{24}$$

Solve the ordinary differential equation:

$$\hat{U}(\omega,t) = \frac{\pi}{\omega}i[\delta(\omega+1)-\delta(\omega-1)]\sin(\omega t)+\pi[\delta(\omega+1)+\delta(\omega-1)]\cos(\omega t) \tag{25}$$

Lastly, we take inverse F.T. and we can get the final solution:

$$u(x,t) = \cos t - x \tag{26}$$

You can click on this link for a common F.T. table.

https://blog.csdn.net/Varalpha/article/details/104964650

2.2 Finite Difference Method

Finite Difference Method (FDM) is a numerical method for solving differential equations by approximating the derivatives with finite difference. For the first-order derivative, we have 3 different forms. Forward difference:

$$\frac{\mathrm{d}y}{\mathrm{d}x}_i = \frac{y_{i+1} - y_i}{\Delta x} \tag{27}$$

Backward difference:

$$\frac{\mathrm{d}y}{\mathrm{d}x}_i = \frac{y_{i+1} - y_i}{\Delta x} \tag{28}$$

Center difference:

$$\frac{\mathrm{d}y}{\mathrm{d}x}_i = \frac{y_{i+1} - y_{i-1}}{2\Delta x} \tag{29}$$

We can approximate second-order derivatives by the center difference of the first-order derivative.

$$\frac{\mathrm{d}^2y}{\mathrm{d}x^2}_i = \frac{y_{i+1} + y_{i-1} - 2y_i}{\Delta x^2} \tag{30}$$

Let's look at the SEIDR Model we got before and try to solve it by programming. I will program with both Python and MATLAB in this paper.

```
import numpy as np
import matplotlib.pyplot as plt

% Parameters
\beta_1 = 0.3  # Infection rate due to I
\beta_2 = 0.4  # Infection rate due to E
\sigma = 0.1   # Incubation rate
\gamma = 0.2   # Recovery rate
\ = 0.     # Mortality rate

% Initial values
S0 = 0.9
E0 = 0.1
I0 = 0.0
D0 = 0.0
R0 = 0.0

% Time parameters
T = 100
dt = 0.1
N = int(T / dt) + 1
t = np.linspace(0, T, N)

% Arrays to store results
S = np.zeros(N)
E = np.zeros(N)
I = np.zeros(N)
D = np.zeros(N)
```

```
R = np.zeros(N)

% Initial conditions
S[0] = S0
E[0] = E0
I[0] = I0
D[0] = D0
R[0] = R0

% Finite difference method
for n in range(N-1):
    S[n+1] = S[n] - beta_1 * S[n] * I[n] * dt - beta_2 * S[n] * I[n] * dt
    E[n+1] = E[n] + beta_1 * S[n] * I[n] * dt + beta_2 * S[n] * I[n] * dt - \
        sigma * E[n] * dt
    I[n+1] = I[n] + sigma * E[n] * dt - (gamma + mu) * I[n] * dt
    D[n+1] = D[n] + mu * I[n] * dt
    R[n+1] = R[n] + gamma * I[n] * dt

% Plot results
plt.figure()
plt.plot(t, S, label='Susceptible')
plt.plot(t, E, label='Exposed')
plt.plot(t, I, label='Infectious')
plt.plot(t, D, label='Dead')
plt.plot(t, R, label='Recovered')
plt.xlabel('Time')
plt.ylabel('Proportion of Population')
plt.legend()
plt.show()
```

2.3 Balance Point and Stability

The behavior differential equations system depends on the boundary and initial conditions. Sometimes the values of a system tend to infinity or oscillate, in other words, the system diverges. In order to get a meaningful solution, we need to confirm the system converging. Recall the population model of Lampreys. The behavior of this system is determined by two values, populations of Lampreys and A (U and V). We can simulate it and plot its state in U-V plane.

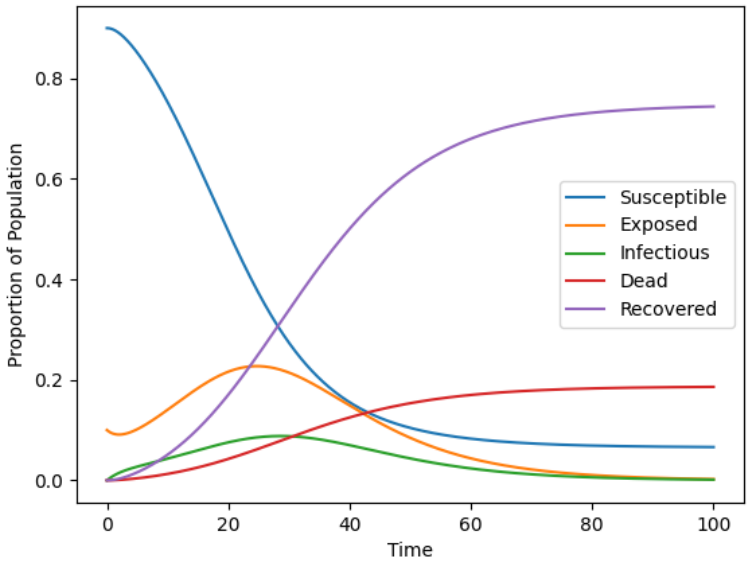

Figure 4: Result of SEIDR Simulation

```python
import numpy as np
import matplotlib.pyplot as plt

# Model parameters
a = 0.08  # Female mortality rate
b = 0.04  # Male competition on females
c = 0.08  # Male mortality rate
d = 0.01  # Female competition on males
rf = 0.2  # Female birth rate
rm = 0.2  # Male birth rate
l = 0.05

# Initial conditions
M_0 = 5  # Initial number of males
F_0 = 5  # Initial number of females
P_0 = M_0 + F_0  # Initial total population
C_0 = 10
N_0 = 100
R = 1.5  # Initial resource availability

# Simulation time parameters
```

```
dt = 0.1  # Time step
t_max = 100  # Simulation time
num_steps = int(t_max / dt) + 1

# Initialize arrays
time = np.linspace(0, t_max, num_steps)
M = np.zeros(num_steps)
F = np.zeros(num_steps)
P = np.zeros(num_steps)
C = np.zeros(num_steps)
N = np.zeros(num_steps)
SexRatio = np.zeros(num_steps)

M[0] = M_0
F[0] = F_0
P[0] = P_0
C[0] = C_0
N[0] = N_0
SexRatio[0] = M_0/F_0


# Solve differential equations
plt.figure(figsize=(10, 6))
plt.plot(M[0], F[0], 'ko', label='State')  # Initial state
for i in range(1, num_steps):
    dF_dt = rf * F[i - 1] * (1 - a * F[i - 1] - b * M[i - 1])
    dM_dt = rm * M[i - 1] * (1 - c * M[i - 1] - d * F[i - 1])
    dP_dt = dM_dt + dF_dt
    dC_dt = (-P[i - 1] * l + R) * C[i - 1]

    M[i] = M[i - 1] + dt * dM_dt
    F[i] = F[i - 1] + dt * dF_dt
    P[i] = P[i - 1] + dt * dP_dt
    C[i] = C[i - 1] + dt * dC_dt

    SexRatio[i] = M[i] / F[i]

    plt.plot(M[i], F[i], 'b.')  # Phase trajectory point
plt.xlabel('Male Population')
plt.ylabel('Female Population')
plt.legend()
```

```
plt.title('Phase Trajectory of the Differential Equation System')
plt.show()

# Plot results
plt.figure(figsize=(10, 6))

# Plot Male Population
plt.plot(time, M, label='Male Population', color='blue')

# Plot Female Population
plt.plot(time, F, label='Female Population', color='red')

# Plot Total Population
plt.plot(time, P, label='Total Population', color='green')

# Additional settings
plt.xlabel('Time')
plt.ylabel('Population')
plt.legend()
plt.title('Population Dynamics of Sea Lampreys')
plt.show()
```

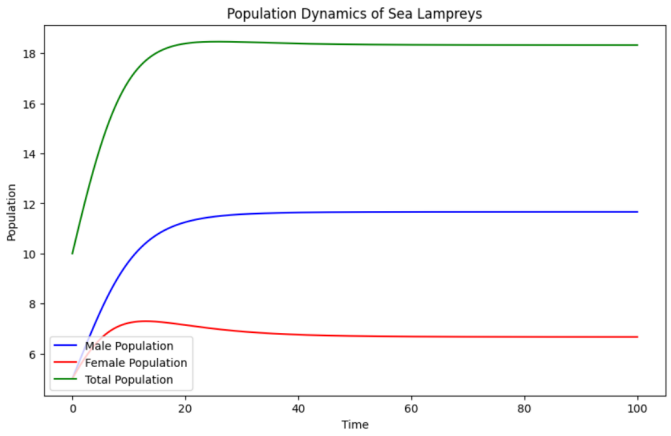

Figure 5: Simulation Result

Figure 6 is the Phase Trajectory Figure of the differential equations system. It reflects the behavior of the two-order system intuitively. With the aid of it, it's much more easy to explain the stability of dynamic systems.
Lyapunov stability is a concept in the field of dynamical systems and control

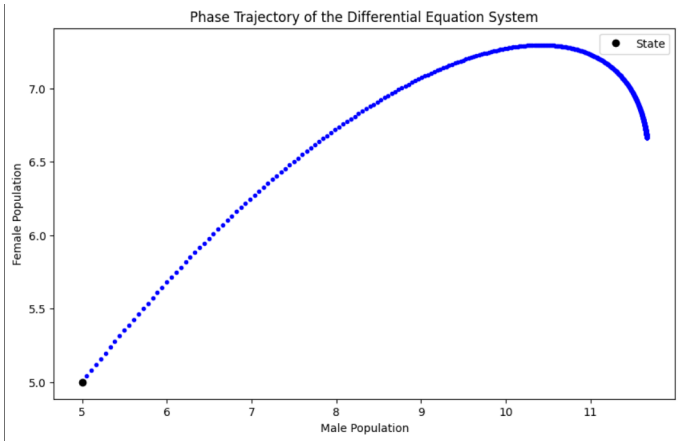

Figure 6: Phase Trajectory

theory that is used to analyze the stability of equilibrium or fixed points in a system. In dynamical systems, an equilibrium point is considered stable if small perturbations from that point do not cause the system to move too far away. Lyapunov stability provides a rigorous mathematical framework to determine whether an equilibrium point is stable, asymptotically stable (eventually going to the equilibrium point), or unstable (diverging away from the equilibrium point). It's defined in mathematics as follows.

Theorem 2.1 *For any t_0 and $\epsilon > 0$, if there is a $\delta(\epsilon, t_0) > 0$ such that if $||x(t, x, t_0)|| < \epsilon$, and $t > t_0$, then the phase trajectory is bounded and the system is in* ***Liapunov Stabilization****.*

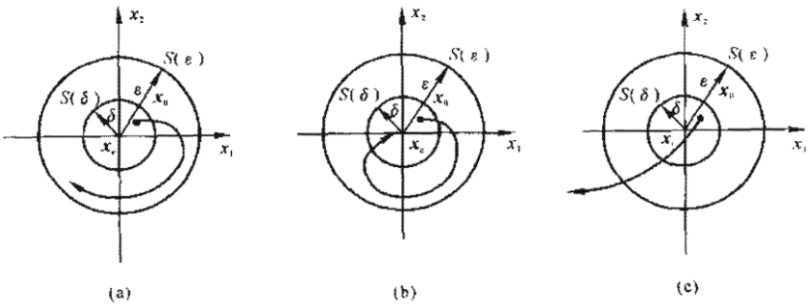

图 2.1 有关稳定性的平面几何表示

(a) 李雅普诺夫意义下的稳定性； (b) 渐近稳定性； (c) 不稳定性

Figure 7: Phase Trajectory in Different Conditions

It's important to point out that this is an extremely simplified version. Liapunov's theory is complicated and impossible to introduce all of them. If

you want to learn more, you can click on the link below.

https://zhuanlan.zhihu.com/p/675832852

2.4 Partial Differential Equation

Differential equations refer to equations that involve the relationship between an unknown function and its derivatives, while partial differential equations involve partial derivatives of unknown functions.
Partial differential equations can describe various natural and engineering phenomena, serving as the primary means of constructing mathematical models in scientific, engineering, and other fields. practical problems in science and engineering can be reduced to boundary value problems of partial differential equations, such as wave propagation, fluid flow and diffusion, vibrations, solid mechanics, electromagnetics, quantum mechanics, and more. There are three main types of partial differential equations: elliptic equations, parabolic equations, and hyperbolic equations.

- Hyperbolic equations describe variables propagating in a certain direction at a given speed, commonly used for describing vibration and wave problems.
- Elliptic equations describe variables spreading to a certain depth in all directions, commonly used for describing static problems like electric fields and gravity fields.
- Parabolic equations describe variables propagating downstream, commonly used for describing transient problems like heat conduction and diffusion.

Solving boundary value problems of partial differential equations is often challenging to obtain an analytical solution, requiring approximate solutions through numerical computing methods. Common numerical methods for partial differential equations include finite difference methods, finite element methods, finite volume methods, conjugate gradient methods, physics-informed machine learning, and more. Typically, the problem domain is first discretized into a grid, and the boundary value problem is then discretized into algebraic equation systems to find approximate values at the discrete grid points. I will use the Finite Element Method to analyze three examples.

2.4.1 Heat Conduction Equation

The heat conduction equation describes the change in temperature within a region over time and is a typical parabolic partial differential equation, also known as the diffusion equation.
The one-dimensional heat conduction equation considers an isotropic and homogeneous thin rod where the temperature is uniform across the cross-section perpendicular to the x-axis, and there is no heat exchange between the rod's circumference and the surrounding environment. In the absence of a heat source within the rod, the temperature u = u(t, x) is a function of the time variable t and the spatial variable x along the horizontal direction.

$$\frac{\partial U}{\partial t} = \lambda \nabla^2 U = \frac{\partial^2 U}{\partial x^2} \tag{31}$$

Here, λ represents the thermal diffusivity, which depends on the material's thermal conductivity, density, and specific heat capacity.

```
import numpy as np
import matplotlib.pyplot as plt

# 2. One-dimensional heat conduction equation (parabolic partial differential e
# pu/pt = l*p2u/px2

# Model parameters
L = 1.0  # Length of the thin rod
lam = 1.0  # Thermal diffusivity
tc = 0  # Start time
te = 10.0  # End time: (0, te)

# Initialization
dx = 0.05  # Spatial step size
dt = 0.001  # Time step size
nNodes = round(L/dx)  # Number of spatial grid nodes
nSteps = round(te/dt)  # Number of time grid nodes
K = lam * dt/(dx**2)  # lambda * dt/dx^2
U = np.zeros([nNodes+1, nSteps+1])  # Create a 2D array

# Boundary conditions
for j in np.arange(0, nSteps+1):  # Time sequence
    U[0,j] = 7.5 + (nSteps-j)/2000 * np.sin(j/1000)/(1+np.exp(-j))
    U[nNodes,j] = 0.0  # Boundary conditions at each time point

```

```
# Initial condition
for i in np.arange(0, nNodes):  # Spatial sequence
    # U[i,0]= 0.2*i*h*(L-i*h)  # Initial condition as a function of x
    U[i,0]= 0  # Initial condition at each position

# Solving with temporal differencing method
for j in np.arange(0, nSteps):  # Time step
    for i in np.arange(1, nNodes):  # Spatial step
        U[i,j+1] = K*U[i+1,j] + (1-2*K)*U[i,j] + K*U[i-1,j]

# Plotting
xZone = np.arange(0, (nNodes+1)*dx, dx)  # Create spatial grid
tZone = np.arange(0, (nSteps+1)*dt, dt)  # Create temporal grid
fig = plt.figure(figsize=(10, 6))
rect1 = [0.05, 0.2, 0.4, 0.65]  # [left, bottom, width, height], 0.0~1.0
ax1 = plt.axes(rect1)
for k in range(0,nSteps+1,round(nSteps/5)):
    ax1.plot(xZone, U[:,k], label=r"x={}".format(k/nSteps))
ax1.set_ylabel('u(x,t)')
ax1.set_xlabel('x')
ax1.set_xlim(0,L)
ax1.set_ylim(-1,12)
ax1.set_title("Temperature distribution along t-axis")
ax1.legend(loc='upper right')
rect2 = [0.55, 0.2, 0.4, 0.65]  # [left, bottom, width, height], 0.0~1.0
ax2 = plt.axes(rect2)
for k in range(0,nNodes+1,round(nNodes/5)):  # U[nNodes,k] = 0.0
    ax2.plot(tZone, U[k,:], label=r"t={}".format(k/nNodes))
ax2.set_ylabel('u(x,t)')
ax2.set_xlabel('t')
ax2.set_xlim(0,te)
ax2.set_ylim(-1,12)
ax2.set_title("Temperature distribution along x-axis")
ax2.legend(loc='upper right')
plt.show()
```

2.4.2 Wave Equations

The wave equation is a typical hyperbolic partial differential equation widely used in fields such as acoustics, electromagnetics, and fluid mechanics to describe various wave phenomena in nature, including transverse waves and

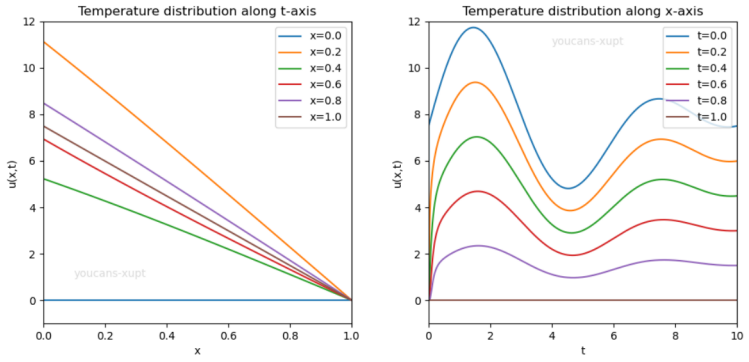

Figure 8: Results of Heat Conduction Equation

longitudinal waves, such as sound waves, light waves, and water waves.
Consider the initial boundary value problem for the following two-dimensional wave equation:

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \\ \frac{\partial u}{\partial t}(0, x, y) = 0 \\ u(x, y, 0) = u_0(x, y) \\ u(0, y, t) = u_a(t), \quad u(1, y, t) = u_b(t) \\ u(x, 0, t) = u_c(t), \quad u(x, 1, t) = u_d(t) \\ 0 \le t \le t_e, \quad 0 < x < 1, \quad 0 < y < 1 \end{cases} \tag{32}$$

where u is the amplitude; c is the wave propagation speed, which can be a fixed constant, a function of position c(x,y), or a function of the amplitude c(u). In this example, the initial condition is given as $u(x, 0) = \sin(6\pi x) + \cos(4\pi y)$, and the boundary conditions are $u(0, y, t) = u(1, y, t) = 0)$ and $u(x, 0, t) = u(x, 1, t) = 0$. Here, the domain is t $(0, 1.0)$ and the spatial domain is $x \in (0, 1.0), y \in (0, 1.0)$. The numerical solution to this problem gives the vibrational state.
To ensure the convergence of the algorithm, the three-point difference format of the upward method requires a step ratio of less than 1:

$$r = \frac{4c^2 \Delta t^2}{\Delta x^2 + \Delta y^2} < 1 \tag{33}$$

Here's the example program.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
```

```
# Model parameters
c = 1.0  # Wave propagation speed
tc, te = 0.0, 1.0  # Time range, 0<t<te
xa, xb = 0.0, 1.0  # Spatial range, xa<x<xb
ya, yb = 0.0, 1.0  # Spatial range, ya<y<yb

# Initialization
c2 = c*c  # Equation parameter
dt = 0.01  # Time step
dx = dy = 0.02  # Space step
tNodes = round(te/dt)  # Number of time grid points on t-axis
xNodes = round((xb-xa)/dx)  # Number of space grid points on x-axis
yNodes = round((yb-ya)/dy)  # Number of space grid points on y-axis
tZone = np.arange(0, (tNodes+1)*dt, dt)  # Build time grid
xZone = np.arange(0, (xNodes+1)*dx, dx)  # Build space grid
yZone = np.arange(0, (yNodes+1)*dy, dy)  # Build space grid
xx, yy = np.meshgrid(xZone, yZone)
# Generate coordinates of grid points xx,yy (2D arrays)

# Courant number check (if r>1, the algorithm is unstable)
r = 4 * c2 * dt*dt / (dx*dx+dy*dy)
print("dt = {:.2f}, dx = {:.2f}, dy = {:.2f}, r = {:.2f}".format(dt,dx,dy,r))
assert r < 1.0, "Error: r>1, unstable step ratio of dt2/(dx2+dy2)."
rx = c*c * dt**2/dx**2
ry = c*c * dt**2/dy**2

# Plotting
fig = plt.figure(figsize=(8,6))
ax1 = fig.add_subplot(111, projection='3d')

# Calculate initial values
U = np.zeros([tNodes+1, xNodes+1, yNodes+1])  # Establish a 3D array
U[0] = np.sin(6*np.pi*xx)+np.cos(4*np.pi*yy)  # U[0,:,:]
U[1] = np.sin(6*np.pi*xx)+np.cos(4*np.pi*yy)  # U[1,:,:]
surf = ax1.plot_surface(xx, yy, U[0,:,:], rstride=2, cstride=2,
    cmap=plt.cm.coolwarm)

# Finite difference method to solve
for k in range(2,tNodes+1):
    if surf:
        ax1.collections.remove(surf)  # Update the 3D animation window
```

```
45
46      for i in range(1,xNodes):
47          for j in range(1,yNodes):
48              U[k,i,j] = rx*(U[k-1,i-1,j]+U[k-1,i+1,j]) + ry*(U[k-1,i,j-1]\
49                  + U[k-1,i,j+1]) + 2*(1-rx-ry)*U[k-1,i,j] -U[k-2,i,j]
50
51      surf = ax1.plot_surface(xx, yy, U[k,:,:],
52          rstride=2, cstride=2, cmap='rainbow')
53      ax1.set_xlim3d(0, 1.0)
54      ax1.set_ylim3d(0, 1.0)
55      ax1.set_zlim3d(-2, 2)
56      ax1.set_title("2D wave equation (t= %.2f)" % (k*dt))
57      ax1.set_xlabel("x-axis")
58      ax1.set_ylabel("y-axis")
59      plt.pause(0.01)
60
61  plt.show()
```

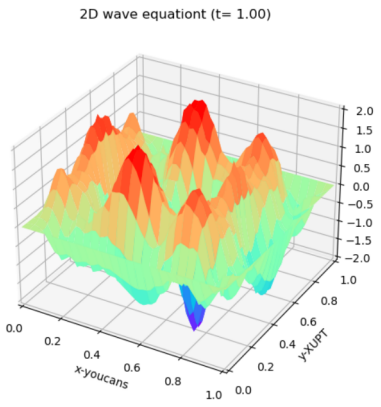

Figure 9: Results of Wave Equation

2.4.3 Poison Equation

The elliptic partial differential equation is an important type of partial differential equation used primarily to describe the equilibrium stable state in physics, such as the electromagnetic field, gravitational field, reaction-diffusion phenomena in a steady state, and is widely applied in fluid mechanics, elasticity, electromagnetics, geometry, and variational methods.

Consider the following two-dimensional Poisson equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \tag{34}$$

The above equation reduces to the Laplace equation when $f(x, y) = 0$. For the numerical solution of two-dimensional elliptic partial differential equations, we use the finite difference method for solving. Simply put, we use a five-point differencing scheme to represent the discrete expression of the second-order derivatives, discretizing the partial differential equation into a finite difference equation:

$$\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta h^2} + \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{\Delta h^2} = f_{i,j} \tag{35}$$

Elliptic partial differential equations describe equilibrium states that do not change with time and do not have initial conditions, so they cannot be solved by iterating along the time step. The above difference equation can be solved using matrix inversion methods, but when the grid is fine with a small Δh, memory consumption and computation requirements for matrix inversion become significant. Therefore, iterative relaxation methods can be used to recursively obtain the numerical solution of the two-dimensional elliptic equation.

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Solution range
xa, xb = 0.0, 1.0  # Spatial range, xa<x<xb
ya, yb = 0.0, 1.0  # Spatial range, ya<y<yb

# Initialization
h = 0.01  # Spatial step size, dx = dy = 0.01
w = 0.5  # Relaxation factor
nodes = round((xb-xa)/h)  # Number of space grid points on x-axis

# Boundary conditions
u = np.zeros((nodes+1, nodes+1))
# u[:, 0] = 0
# u[:, -1] = 0  # -1 represents the last value in the array
# u[0, :] = 1
# u[-1, :] = 1  # -1 represents the last value in the array
```

```
for i in range(nodes+1):
    u[i, 0] = 1.0 + np.sin(0.5*(i-50)/np.pi)
    u[i, -1] = -1.0 + 0.5*np.sin((i-50)/np.pi)
    u[0, i] = -1.0 + 0.5*np.sin((i-50)/np.pi)
    u[-1, i] = 1.0 + np.sin(0.5*(50-i)/np.pi)

# Solve using iterative relaxation method
for iter in range(100):
    for i in range(1, nodes):
        for j in range(1, nodes):
            u[i, j] = w/4 * (u[i-1, j] + u[i+1, j] +
                u[i, j-1] + u[i, j+1]) + (1-w) * u[i, j]

# Plotting
x = np.linspace(0, 1, nodes+1)
y = np.linspace(0, 1, nodes+1)
xx, yy = np.meshgrid(x, y)
fig = plt.figure(figsize=(8,))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(xx, yy, u, cmap=plt.get_cmap('rainbow'))
fig.colorbar(surf, shrink=0.5)
ax.set_xlim3d(0, 1.0)
ax.set_ylim3d(0, 1.0)
ax.set_zlim3d(-2, 2.5)
ax.set_title("2D elliptic partial differential equation")
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
plt.show()
```

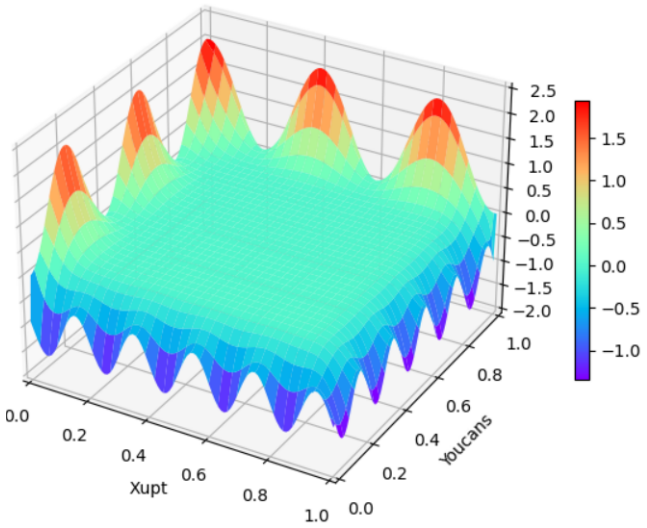

Figure 10: Results of Poison Equation